



HOCHSCHULE LANDSHUT

FAKULTÄT ELEKTROTECHNIK UND
WIRTSCHAFTSINGENIEURWESEN

Studiengang Elektrotechnik (M. Eng.)

Projektarbeit eingebettete Systeme (Medizintechnik)

**Entwicklung einer myoelektrisch gesteuerten
Handprothese**

**Weiterentwicklung und Gesten-Erkennung mittels
Machine Learning**

Vorgelegt von: Felix Kraye

Eingereicht am: 08.07.2026

Betreuer: Prof. Dr. Andreas Breidenassel

Erklärung zur Projektarbeit

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benützt, sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Ich versichere, dass ich von mir verwendete KI-Werkzeuge als Hilfsmittel angegeben und mit ihrem Produktnamen und einer Übersicht der im Rahmen dieser Prüfungsarbeit genutzten Eingabeparameter (Prompts) im Verzeichnis „Übersicht verwendeter Hilfsmittel“ vollständig aufgeführt habe. Sämtliche mittels KI-Werkzeugen generierten Inhalte und Ergebnisse wurden von mir als solche kenntlich gemacht.

Ort, Datum

Unterschrift

Danksagung

Ein ganz besonderer Dank gilt meiner Kommilitonin Isabell Zeitler. Sie hat das Projekt im Vorsemester ins Leben gerufen und mir den Entwicklungsstand bei der Übergabe im Detail erklärt. Darüber hinaus hat sie die diessemestrige Weiterentwicklung kontinuierlich mit wertvollen Ideen und konstruktiven Vorschlägen bereichert. Mit ihrem fundierten Wissen stand sie mir wie ein festes Teammitglied zur Seite. Ihr maßgebliches Mitwirken bei der Kontrolle des Layouts und der Inbetriebnahme der neuen Platine hat entschieden zum Erfolg des Projekts beigetragen

Inhaltsverzeichnis

Abkürzungsverzeichnis	II
1 Einleitung	1
2 Zielsetzung	1
3 Hardware- und Elektronikentwicklung	2
3.1 Erweiterung der sEMG-Kanäle und Platinen-Redesign	2
3.1.1 Bestehender Grundaufbau	3
3.1.2 Optimierungen aus den Erkenntnissen des Vorsemesters	3
3.1.3 Kanalerweiterung durch Wechsel auf den ADS1298	4
3.1.4 Modifikation der Steckverbindungen	4
3.1.5 Inbetriebnahme	4
3.2 Konstruktion der Elektroden-Armbänder	5
4 Softwarearchitektur und Signalverarbeitung	6
4.1 Datenakquisition und Firmware-Anpassungen	6
4.2 Digitale Filterung auf dem Mikrocontroller	7
5 Mustererkennung und Machine Learning	7
5.1 Evaluierung öffentlicher EMG-Datensätze und Transfer Learning-Ansatz	8
5.2 Methodik der eignen Datenerfassung	9
5.3 Feature-Selektion, Modellwahl und Training am PC	10
5.4 Firmware-Implementierung des Klassifikationsmodells	12
6 Evaluierung und Ergebnisse	13
6.1 Systemlatenz und Echtzeit-Verhalten	13
6.2 Biomechanische Signalcharakteristika und Besonderheiten	14
6.3 Robustheit gegenüber Sensor-Repositionierung (Spatial Shift)	14
7 Zusammenfassung und Ausblick	15
7.1 Zukünftige Optimierungs- und Erweiterungspotenziale	16
7.2 Probanden-Kompatibilität und mobiles Training	16

Abkürzungsverzeichnis

sEMG Oberflächen-Elektromyographie

EMG Elektromyographie

EKG Elektrokardiogramm

ML Maschinelles Lernen

ML Machine Learning

RLD Right Leg Drive

AFE Analog-Front-End

SPI serielle Peripherieschnittstelle

RMS Root Mean Square

MAV Mean Absolute Value

1 Einleitung

Der Verlust einer Hand durch eine Amputation oder aufgrund einer angeborenen Fehlbildung bedeutet für die betroffenen Personen einen tiefgreifenden Einschnitt in den Alltag und eine erhebliche Einschränkung der Lebensqualität. In der modernen Medizintechnik zeigt sich in den letzten Jahrzehnten eine deutliche Weiterentwicklung von rein kosmetischen Passiv-Prothesen oder Eigenkraftprothesen hin zu aktiven, mechatronisch angetriebenen Prothesen [2]. Das Ziel liegt heute in der Entwicklung intelligenter Systeme, die eine möglichst intuitive und feingliedrige Interaktion zwischen Mensch und Maschine erlauben.

Als Schnittstelle für die Erfassung von Biosignalen hat sich sowohl in der Forschung als auch in modernen kommerziellen Anwendungen die Oberflächen-Elektromyographie (**sEMG**) etabliert [1, 3, 6]. Bei diesem nicht-invasiven Verfahren werden die schwachen elektrischen Potenziale, die bei einer Kontraktion der Skelettmuskulatur entstehen, mithilfe von Elektroden direkt auf der Hautoberfläche gemessen. Diese Signale spiegeln direkt die Bewegungsabsicht des Anwenders wider. Eine der größten technologischen Herausforderungen besteht jedoch darin, dass **sEMG**-Signale extrem anfällig für externe Störungen sind. Sie weisen sehr geringe Amplituden auf und werden im Alltag häufig durch elektromagnetisches Netzbrummen, Bewegungsartefakte der Kabel oder das sogenannte Übersprechen (Crosstalk) benachbarter Muskeln überlagert.

Diese Projektarbeit knüpft direkt an die Ergebnisse des Vorsemesters an [8]. In der vorangegangenen Arbeit wurde bereits ein funktionsfähiger Prothesen-Prototyp auf Basis des Open-Source-Modells *inMoov* als 3D-Druck realisiert und mit Seilzügen und Servomotoren ausgestattet. Zudem wurde eine eigens entwickelte Hardware-Platine in Betrieb genommen, die eine präzise Aufzeichnung der Muskelaktivitäten über vier Kanäle ermöglichte. Die darauf aufbauende Steuerung basierte allerdings auf einfachen amplitudenbasierten Schwellenwerten. Das bedeutet, dass die Prothese lediglich auf allgemeine Signalstärke reagierte, was eine differenzierte Erkennung einzelner Fingerbewegungen und Handgesten nicht ermöglichte. Hier setzt die vorliegende Arbeit an, um das System von einer reinen Schwellenwertsteuerung zu einer intelligenten musterorientierten Erkennung weiterzuentwickeln.

2 Zielsetzung

Das übergeordnete Ziel dieser Projektarbeit ist die sensorische Erweiterung sowie die softwareseitige Weiterentwicklung der bestehenden myoelektrischen Handprothese. Durch den Einsatz von Maschinelles Lernen (**ML**) direkt auf dem eingebetteten System (Edge AI) soll eine intuitive und verlässliche Erkennung verschiedener Handgesten in Echtzeit ermöglicht werden.

Um dieses Kernziel zu erreichen, gliedert sich das Projekt in folgende wesentlich Teilaufgaben:

- **Hardware-Optimierung und Kanalerweiterung:** Um komplexere Muskelakti-

vitätsmuster im Unterarm räumlich feiner auflösen zu können, wird das bestehende Messsystem von vier auf insgesamt acht **sEMG**-Kanäle erweitert. Zudem wird eine mechanische Vorrichtung in Form eines Sensor-Armbands entwickelt. Diese soll gewährleisten, dass die Elektroden äquidistant, reproduzierbar und mit konstantem Anpressdruck am Arm des Anwenders platziert werden können. Dadurch soll die Abweichung der Elektrodenpositionen beim Neuanlegen minimiert werden, damit das System reproduzierbare Muskelaktivitätsmuster liefert.

- **Echtzeit-Signalverarbeitung auf dem Mikrocontroller:** Auf dem eingebetteten System wird eine performante Software-Pipeline implementiert. Diese soll die **sEMG**-Signale bei einer Abtastrate von 1000 Hz online filtern, um Störungen wie das allgegenwärtige Netzbrummen oder Bewegungsartefakte zu eliminieren. Aus den bereinigten Datenströmen werden kontinuierlich Kennwerte extrahiert, die als mathematische Merkmalsvektoren für die anschließende Mustererkennung dienen. Im Anschluss werden die Motoren angesteuert um die erkannte Geste auf der Prothese umzusetzen.
- **Mustererkennung mittels Machine Learning (Edge AI):** Statt starrer Grenzwerte wird ein maschinelles Lernverfahren für die Gesten-Erkennung eingesetzt. Hierzu wird in einer vorgelagerten Trainingsphase ein Klassifikationsmodell auf Basis von aufgezeichneter **sEMG**-Daten trainiert. Diese Daten wurden zuvor mit dem selben System erstellt. Das Modell wird anschließend so in die Firmware eingebettet, dass die Echtzeit-Klassifikation der Merkmalsvektoren im autarken Betrieb auf dem Mikrocontroller ausgeführt werden kann.
- **Evaluation der Gesten-Erkennung:** Das Gesamtsystem wird anschließend empirisch evaluiert. Hierbei steht die Klassifikationsgenauigkeit im Fokus. Es wird insbesondere untersucht, wie sich ein Vortraining mit einem bestehenden Datensatz (Myo Dataset [4]) auswirkt und in welchem Maße das System tolerant gegenüber dem Abnehmen und Wiederanlegen des **sEMG**-Armbands (Sensor-Repositionierung) ist.

3 Hardware- und Elektronikentwicklung

Die Zuverlässigkeit einer myoelektrischen Prothesensteuerung hängt maßgeblich von der Qualität der erfassten Analogsignale ab. Da sich die Amplituden von **sEMG**-Signalen im mikro- bis niedrigen Millivoltbereich bewegen, stellt das Design der Leiterplatte hohe Anforderungen an die elektromagnetische Verträglichkeit und die Signalintegrität. In diesem Abschnitt wird die hardwareseitige Überarbeitung und Erweiterung der Prothesenelektronik beschrieben.

3.1 Erweiterung der sEMG-Kanäle und Platinen-Redesign

Das im Vorsemester entwickelte Hardware-Konzept hat sich als verlässliche Basis bewiesen und wurde in seinem Grundaufbau übernommen [8]. Das Primäre Ziel des Redesigns

besteht darin, die Anzahl der Messkanäle von vier auf acht zu erhöhen und bekannte Schwachstellen der ersten Prototypen-Generation zu eliminieren.

3.1.1 Bestehender Grundaufbau

Das Layout der Platine wurde beibehalten. Die Leiterplatte unterteilt sich weiterhin in zwei strikt galvanisch getrennte Bereiche, um die Sicherheit des Nutzers gemäß medizinischen Standards zu garantieren und das Einkoppeln von Schaltregler- und Motorrauschen zu verhindern. Der Aufbau gliedert sich weiterhin wie folgt:

- **Digitale Prothesenseite:** Hier befindet sich der zentrale Mikrocontroller (Teensy 4.1), die Peripherie zur Ansteuerung der Servomotoren sowie die primäre Spannungsversorgung.
- **Isolierte Analogseite:** Dieser Bereich beherbergt den ADS129x als Biosignalverstärker, Analog-Front-End (AFE) und Analog-Digital-Wandler in einem Bauteil. Des Weiteren befinden sich hier die Steckverbindungen zu den sEMG-Elektroden mit den dazugehörigen Eingangsfiltern.
- **Isolationsbarriere:** Die Trennung wird hardwareseitig durch zwei digitale Isolatoren realisiert, über die die SPI-Kommunikation des ADS129x mit dem Teensy 4.1 stattfindet. Des Weiteren fungiert einer der beiden Isolatoren als DC/DC-Wandler um die 5V Eingangsspannung auf die 3,3V der Analogseite zu übertragen.

3.1.2 Optimierungen aus den Erkenntnissen des Vorsemesters

Bei der Überarbeitung des Layouts wurden zwei wesentliche Detailverbesserungen vorgenommen, die unmittelbar aus den praktischen Erfahrungen der vorherigen Inbetriebnahme basieren.

Zum einen betrifft dies die Korrektur des Versorgungslayouts durch die vollständige Eliminierung eines fehleranfälligen Jumpers. Im vorherigen Entwurf existierte ein Designfehler im Bereich des digitalen Isolators, bei dem ein dort platzierter 3,3 V-Jumper bei Entfernen die Spannungsversorgung der isolationsinternen Logik unterbrach. Dies resultierte ausgangseitig in einer zu hohen Spannung von 4,2 V sowie einer kritischen thermischen Belastung des Bauteils. Im neuen Layout wurden die entsprechenden Versorgungspins daher im Layout der Leiterplatte fest überbrückt, um diesen Fehler auszuschließen.

Außerdem konnte die analoge Eingangsfilterung erheblich kompaktiert werden, welche als RC-Tiefpass an jedem Messeingang zur Dämpfung von Hochfrequenzstörungen und als Antialiasing-Filter dient. Dabei wurden die als optimal ermittelten Filterwerte von 5,1 k Ω und 1 nF aus dem Vorbericht übernommen. Um den Platzbedarf auf der Platine trotz der verdoppelten Kanalanzahl zu minimieren und gleichzeitig die Bestückung der Platine zu vereinfachen, wurden die zuvor getrennt platzierten Widerstände durch integrierte SMD-Resistor-Arrays ersetzt, welche jeweils vier 5 k Ω -Widerstände in einem Bauteil vereinen.

TODO hier Vergleich zwischen altem und neuen Layout

3.1.3 Kanalerweiterung durch Wechsel auf den ADS1298

Die Erhöhung der Kanalanzahl von vier auf acht Kanäle ließ sich dank der durchdachten Bauteilauswahl des Vorsemesters sehr leichtgewichtig umsetzen. Es wurde von der 4-Kanal-Variante des Biosignalverstärkers (ADS1294) auf die 8-Kanal-Variante (ADS1298) gewechselt. Da beide integrierten Schaltkreise innerhalb ihrer Gehäuse vollständig pin-kompatibel sind, blieb der Anschluss der Versorgungs-, Referenz- und Kommunikationspins identisch.

Hardwareseitig mussten im Layout lediglich die zuvor ungenutzten und auf Masse gelegten differenziellen Eingangspins (pro Kanal jeweils ein positiver P- und ein negativer N-Eingang) an die neuen Elektrodenanschlüsse herangeführt werden. Auch die Softwareanpassung gestaltete sich als sehr leichtgewichtig, da im Wesentlichen nur einzelne Kanalanzahl-Konstante in der Firmware angepasst werden mussten.

3.1.4 Modifikation der Steckverbindungen

Eine wichtige mechanische und elektrische Verbesserung betrifft die Eingangsbuchsen der Elektroden. In der ersten Generation wurden 3,5 mm Klinkenbuchsen verwendet. Diese neigten im Praxisbetrieb zu ungleichmäßiger Signalqualität, wobei es insbesondere es massive qualitative Unterschiede zwischen verschiedenen Anschlusskabeln gab. Dies äußerte sich in einem inkonsistenten Rauschverhalten.

Im Redesign wurden die Klinkenbuchsen vollständig durch 1,5 mm Anschlusspins ersetzt, die dem Standard bei **EKG**- und **EMG**-Geräten in der medizinischen Diagnostik entsprechen. In der Praxis zeigte sich durch diesen Wechsel eine konsistente Signalqualität über alle Kanäle hinweg. Die kabelabhängige Varianz der Rauschwerte wurde eliminiert, sodass alle acht Kanäle qualitativ das niedrige Grundrauschen erreichen, das zuvor nur mit den hochwertigen Klinkenkabeln erzielt werden konnte.

3.1.5 Inbetriebnahme

Nach der Fertigung und Bestückung der neuen Platine wurde das neue System in Betrieb genommen. Zur schnellen visuellen Funktionskontrolle wurde testweise ein Elektrokardiogramm (**EKG**) aufgenommen, da dessen markanter Signalverlauf eine Zuverlässige Beurteilung der Hardwarefunktionalität erlaubt. Dabei zeigte sich ein massives, breitbandiges Rauschen, welches das eigentliche Biosignal vollständig überlagerte. Als potentielle Ursache wurden Potentialverschiebungen des menschlichen Körpers gegenüber der analogen Masse **AGND** identifiziert. Da der Körper wie eine Antenne wirkt, koppelt er gegebenenfalls starke Störfrequenzen aus der Umgebung ein, wie beispielsweise das 50Hz Netzbrummen.

Die Lösung dieses Problems lag darin, den bereits implementierten Right Leg Drive (**RLD**) des ADS1298 mit der analogen Masse **AGND** zu verbinden. Diese Schaltung greift

das Gleichtaktsignal der Messkanäle ab, invertiert es und speist es aktiv über eine Referenzelektrode zurück in den Körper ein, um die Störungen destruktiv zu überlagern. Da der **RLD**-Kreis jedoch zuvor keinen direkten, stabilen Bezug zu AGND aufwies, konnte er die Potentialverschiebungen nicht optimal ausgleichen. Die Lösung wurde durch das Setzen einer Drahtbrücke realisiert, welche auf den AGND-Pin gesteckt und an der anderen Seite auf das **RLD**-Testfeld gelötet wurde. Mit dieser Anpassung stabilisierte sich das System, wodurch das Breitbandrauschen vollständig und dauerhaft eliminiert werden konnte. Damit bildet das System eine solide Basis für fehlerfreie Signalaufzeichnung im späteren Betrieb.

3.2 Konstruktion der Elektroden-Armbänder

Für eine reproduzierbare Gesten-Erkennung ist die mechanische Fixierung der Sensoren auf der Haut von zentraler Bedeutung. Da durch die Erhöhung auf acht differentielle Messkanäle insgesamt 16 einzelne Klebeelektroden sowie eine zusätzliche Elektrode für den **RLD** präzise platziert werden müssten, scheidet ein rein manuelles Aufkleben aus Praktikabilitätsgründen aus. Zu diesem Zweck wurde eine mechanische Konstruktion aus zwei elastischen Elektroden-Armbändern realisiert.

Als Basismaterial dient ein einfaches Gummi-Nahtband mit einer Breite von 3 cm. Daraus wurden per Hand zwei geschlossene Ringe genäht, deren Umfänge an die Anatomie des Unterarms angepasst sind. In diese Gummibänder sind in gleichmäßigen Abständen präzise Einschnitte eingebracht, die als Halterungen für die Elektrodenenden dienen.

Als Kontaktelemente kommen medizinische Einweg-Klebeelektroden zum Einsatz, bei denen vor der Anbringung der Klebering entfernt wird, um eine reine Trockenelektroden-Messung zu ermöglichen. Da nur die metallischen Komponenten der Klebeelektroden als Kontakte genutzt werden, lassen sich die eigentlichen Einwegelektroden dauerhaft wiederverwenden. Die metallischen Druckknopfenden der Elektroden werden von innen durch die Einschnitte im Gummiband gesteckt und von außen mit dem Druckknopf-Gegenstück des Anschlusskabels mechanisch fixiert. Dabei wird gleichzeitig der elektrische Kontakt hergestellt.

Versuche, die Signalqualität durch den zusätzlichen Einsatz von medizinischem Kontaktgel auf zu verbessern, lieferten negative Ergebnisse. Unter dem Anpressdruck der Armbänder verschmierte das Gel flächig auf der Haut und führte zu Kurzschlüssen zwischen den eng beieinander liegenden Kanälen, was die Signalqualität im Vergleich zu einer reinen Trockenmessung massiv verschlechterte.

Das Gesamtsystem besteht aus zwei separaten Armbändern, mit jeweils acht Elektroden, um die differenzielle Signalaufzeichnung abzubilden. Das P-Kanal-Armband nimmt alle acht positiven Elektrodenanschlüsse auf und wird proximal auf dem Unterarm (näher am Ellenbogen) platziert. Das N-Kanal-Armband hält die acht negativen Anschlüsse, wird distal (näher am Handgelenk) positioniert und ist aufgrund des an dieser Stelle geringeren Armumfangs etwas enger dimensioniert. Die **RLD**-Referenzelektrode wird nach wie vor als einzelne, separate Klebeelektrode unabhängig von den Bändern am Arm positioniert. Eine

Anbringung seitlich der Ellenbogenbeuge hat sich als robuste Positionierung erwiesen.

Die äquidistante Anordnung der Elektroden über den Armumfang wird durch die exakten Abstände der Einschnitte im Gummiband vorgegeben. Ein gleichmäßiger und konstanter Anpressdruck aller acht Elektroden pro Armband wird nach dem Anlegen über die elastische Spannung des Gummibands erreicht. Da sich diese Kraft gleichmäßig über den Armumfang verteilt, gleicht das Band automatisch Unregelmäßigkeiten aus und verhindert ein Verrutschen bei Bewegungen.

4 Softwarearchitektur und Signalverarbeitung

Um die vom Analog-Front-End (**AFE**) erfassten Muskelaktivitäten in präzise Steuerbefehle für die Handprothese zu übersetzen, bedarf es einer performanten Softwarearchitektur auf dem Mikrocontroller. Dieser Abschnitt beschreibt die softwareseitige Implementierung auf dem **Teensy 4.1**. Die Architektur unterteilt sich im Wesentlichen in die hardwarenahe Datenakquisition über serielle Peripherieschnittstelle (**SPI**), die digitale Signalaufbereitung mittels Frequenzfiltern sowie die anschließende Feature-Extraktion und Klassifikation.

4.1 Datenakquisition und Firmware-Anpassungen

Die Basis der Signalverarbeitung bildet die Abfrage des **AFE**. Während im Vorsemester ein **ADS1294** eingesetzt wurde, bei dem firmwareseitig vier Kanäle aktiv ausgelesen wurden und vier über **SPI** gesendete Dummy-Pakete verworfen wurden, erfolgt in der neuen Version der vollständige Umstieg auf den **ADS1298**. Damit einhergehend wurde die zentrale Firmware so erweitert, dass nun alle acht verfügbaren Kanäle parallel erfasst und verarbeitet werden.

Die Kernlogik der Datenaufnahme basiert auf einem interruptgesteuerten System, um eine konstante Abtastrate von $f_S = 1000 \text{ Hz}$ zu garantieren. Sobald das **AFE** eine Wandlung abgeschlossen hat, zieht es die **DRDY**-Leitung (Data Ready) auf *LOW*. Dies löst auf dem **Teensy 4.1** direkt einen Hardware-Interrupt aus, welcher die Abfrage der Daten in der Hauptschleife initiiert. Innerhalb der Schleife werden über den **SPI**-Bus mit einer Frequenz von 1 MHz insgesamt neun Datenpakete mit jeweils 24 Bit (Status-Byte sowie acht Kanal-Bytes) seriell übertragen. Direkt nach dem Einlesen werden die 24-Bit-Werte mittels Sign-Extension in 32-Bit `int32_t` umgewandelt. Anschließend werden die Rohwerte mit einem konstanten Faktor multipliziert, um den Messwert in Mikrovolt (μV) darzustellen:

$$\text{Messwert } (\mu\text{V}) = \text{RawValue} \cdot 0,0397 \mu\text{V}$$

Für die optionale Datenerfassung auf dem PC, sowie die Echtzeitvisualisierung wurde die bereits vorhandene serielle USB-Schnittstelle (betrieben bei 2.000.000 Baud) verwendet. Diese wurde firmwareseitig so skaliert, dass anstelle der ursprünglichen vier Kanäle nun die bereinigten Daten aller acht Kanäle synchronisiert auf der seriellen Schnittstelle ausgegeben werden.

Wie bereits in [Abschnitt 3.1.3](#) erwähnt, beliefen sich die in diesem [Abschnitt 4.1](#) erwähnten Anpassungen gegenüber der Software des Vorsemesters lediglich auf die Erhöhung einzelner Kanalanzahl-Konstanten von vier auf acht.

4.2 Digitale Filterung auf dem Mikrocontroller

Bevor die akquirierten Daten über die Serielle Schnittstelle ausgegeben, oder in einem Ringpuffer für die Klassifikation gespeichert werden, durchlaufen sie eine digitale Filterkaskade um die Rohdaten von Störanteilen zu befreien. Die Ziele bestehen zum einen in der Entfernung des 50 Hz-Netzbrummens sowie dessen Harmonischen und zum anderen in einer Hochpassfilterung, um niederfrequente Bewegungsartefakte zu eliminieren und die Signale um null zu zentrieren. Während im Vorsemester lediglich eine mathematische Baseline-Korrektur mittels eines Hochpassfilters erster Ordnung stattfand, wurde die Signalverarbeitungskette in der aktuellen Firmware um eine kaskadierte Biquad-Filterstruktur erweitert. Die Filterpipeline operiert bei einer Abtastrate von 1000 Hz auf allen acht Kanälen und setzt sich aus den folgenden vier Stufen zusammen:

1. **Hochpassfilter (15 Hz):** Dämpft niederfrequente Bewegungsartefakte.
2. **Notch-Filter (50 Hz):** Eliminiert das dominante Netzbrummen der Umgebungselektronik.
- 3./4. **Notch-Filter (100 Hz & 200 Hz):** Unterdrückt gezielt die ersten beiden harmonischen Oberschwingungen des Netzbrummens.

Die mathematische Umsetzung erfolgt online direkt innerhalb der Datenerfassungsprozedur über die Differenzengleichung eines Direct-Form-I-Biquads:

$$y[n] = b_0 \cdot x[n] + b_1 \cdot x[n-1] + b_2 \cdot x[n-2] + a_1 \cdot y[n-1] + a_2 \cdot y[n-2]$$

Die Effektivität dieser Filterkaskade lässt sich signifikant im Frequenzbereich nachweisen. Wie in [Abbildung 1](#) dargestellt, führt das ungesättigte Rohsignal im Leistungsdichte- und Amplitudenspektrum zu massiven Peaks bei 50 Hz, 100 Hz sowie 200 Hz. Nach der Filterung sind diese Störlinien vollständig eliminiert, während das relevante Frequenzband des Elektromyographie ([EMG](#))-Signals für die Merkmalsextraktion erhalten bleibt.

5 Mustererkennung und Machine Learning

Während in der Projektarbeit des Vorsemesters die Ansteuerung der Endeffektoren über empirisch ermittelte Schwellenwertlogik realisiert wurde, setzt das aktuelle System auf eine intelligente, musterbasierte Erkennung mittels Machine Learning. Die biologischen [EMG](#)-Signale weisen bei identischen Gesten individuelle, aber dennoch reproduzierbare Aktivierungsmuster über die acht sensorischen Kanäle auf. Um diese komplexen Signalcharakteristika robust und in Echtzeit zu interpretieren, werden verschiedene Algorithmen

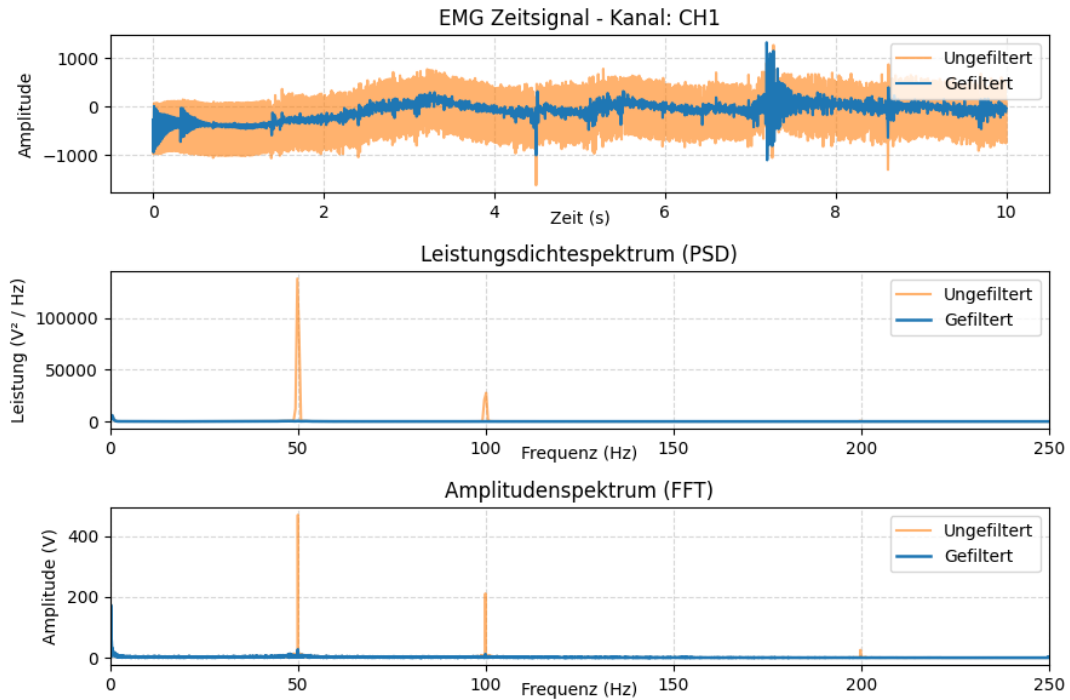


Abbildung 1: Vergleich von ungefiltertem (orange) und gefiltertem (blau) **EMG**-Signal. Gut zu erkennen ist die vollständige Eliminierung des 50 Hz-Netzbrummens sowie dessen harmonischer Oberschwingungen (100 Hz, 200 Hz) im Frequenzbereich.

eingesetzt. Dieser Abschnitt beschreibt den Weg von der Evaluation bestehender Datensätze, über eine eigene Datenerfassungs-Pipeline bis hin zum Training und dem finalen C-Code des Klassifikationsmodells.

5.1 Evaluierung öffentlicher EMG-Datensätze und Transfer Learning-Ansatz

Zu Beginn der Modellentwicklungsphase stand das Ziel, ein robustes Klassifikationsmodell durch den Einsatz bereits existierender, öffentlich zugänglicher EMG-Datensätze vorzutrainieren. Dieser Ansatz wird als *Transfer Learning* bezeichnet und bietet in vielen Fällen den Vorteil, auf einer breiten und generalisierten Datenbasis aufzubauen, um ein besseres Ergebnis zu erzielen, wenn nur wenige eigene Daten vorliegen.

Hierfür wurden die folgenden beiden Datensätze evaluiert

1. **Ninapro-Datenbank [5]:** Diese stellt eine der größten Sammlungen für robotergetstützte Prothesensteuerung dar. Sie schied jedoch früh für das Vortraining aus, da die Datensätze hochkomplexe, dichte Elektrodensetups mit 10-16 Kanälen nutzen. Dies lässt sich mit der vorliegenden Achtkanal-Hardware nicht abbilden. Außerdem beinhalten die Datensätze neben reinen **EMG**-Daten eine Vielzahl weiterer, für dieses Projekt irrelevanter Sensorsignale, die nicht verwendet werden können. Zwar erwies sich ein spezifischer Teildatensatz (DB5) als vielversprechend, da er mit zwei *Thalmic Myo-Armbändern* (jeweils acht Kanäle) aufgezeichnet wurde, war jedoch

aufgrund der Fülle an verschiedenen Handbewegungen für die aktuelle Projektphase zu komplex [7].

2. **Myo-Dataset [4]:** Dieses Open-Source Repository erwies sich als hochgradig kompatibel, da es explizit auf Gestenklassifikation mittels des kommerziellen, achtkanaligen *Thalmic Myo-Armbands* erstellt wurde. Der Aufbau dieses Datensatzes inspirierte maßgeblich die Entscheidung, die Klassifikation in der aktuellen Projektphase zunächst auf die Gesten *Schere*, *Stein*, *Papier* zu fokussieren, anstatt direkt eine komplexe Einzelfingersteuerung anzustreben.

Das *Myo-Dataset* brachte jedoch spezifische Signalcharakteristika mit sich, die für die eigene Implementierung einer Klassifikations-Pipeline von Bedeutung waren. Die Daten des Myo-Armbands liegen herstellerbedingt mit einer Abtastrate von 200 Hz vor und sind fest auf einen Wertebereich von $[-128, 127]$ normiert (8-Bit-Integer). Um ein eigenes Modell auf Basis dieser Daten zu trainieren und anschließend mit dem eigenen Hardware-Setup zu testen, mussten diese Parameter zwingend in die eigne Signalverarbeitung adaptiert werden.

Das Domain Gap-Problem: Ein Versuch des Transfer-Learnings offenbarte jedoch die Grenzen dieses Ansatzes. Das auf den über 1000 öffentlichen Datenpunkten selbst trainierte Modell erreichte bei der anschließenden Validierung auf dem eignen Hardware-Setup eine Genauigkeit von etwa 33 %. Bei drei zu unterscheidenden Gesten entspricht dies genau der Ratewahrscheinlichkeit. Die Ursache hierfür liegt in einem klassischen Domain Gap: Die andere Sensorik des ADS1298, die individuelle Elektrodenplatzierung auf dem Unterarm, sowie die abweichende Signalverarbeitung führten zu völlig anderen Signalcharakteristika als die des originalen Myo-Armbands.

Da das Modell auf den eigenen Realdaten versagte, während es auf dem Abgetrennten Testanteil des *Myo-Datasets* über 90 % erzielte, wurde die Strategie des Vortrainings verworfen. Es wurde deutlich, dass für ein verlässliches Gesamtsystem die Generierung eines eigenen, hardware- und Probandenspezifischen Datensatzes unumgänglich war.

5.2 Methodik der eignen Datenerfassung

Da die Nutzung externer Daten fehlschlug, wurde eine eigene Datenerfassungs-Pipeline implementiert. Ziel war es, einen konsistenten, balancierten und korrekt gelabelten Datensatz auf der Basis der eigenen Hardware aufzubauen. Zu diesem Zweck wurde ein Python-Skript entwickelt, dessen Ablaufstruktur stark an das Design des *Myo-Datasets* [4] angelehnt ist. Dadurch wurde eine strukturierte und reproduzierbare Aufnahmeumgebung geschaffen.

Das Skript steuert den Aufnahmezyklus interaktiv über das Terminal-Interface und bildet folgenden Ablauf ab:

1. **Eingabe der Aufnahmemenge:** Vor dem Start der Messungen wird die gewünschte Anzahl der aufzunehmenden Gesten-Tripel (Durchgänge) definiert. Innerhalb je-

des Tripels werden alle drei Gesten (*Schere, Stein, Papier*) exakt einmal aufgenommen. Die Aufnahmen erfolgen durch Wiederholungen der Schritte 3. und 4..

2. **Randomisierung:** Um serielle Übergangseffekte zwischen den Gesten zu verhindern (z.B. bei der Aufnahme folgt *Stein* immer auf *Papier*) und eine unverfälschte Repräsentation der Gesten sicherzustellen, wird die Gestenreihenfolge innerhalb jedes Tripels per Zufallsalgorithmus neu bestimmt. Zudem hält die unvorhergesehene Abfolge das Aufmerksamkeitslevel des Probanden über die gesamte Messdauer konstant und verzögert Ermüdungseffekte.
3. **Vorbereitungspahse:** Das Skript kündigt die als Nächstes einzunehmende Geste im Terminal an (z.B. `Prepare for ROCK`). Dieses Zeitfenster dauert eine Sekunde und dient dem Probanden gleichzeitig als Entspannungsphase für die Unterarmmuskulatur.
4. **Aktivierungsphase:** Nach einer Sekunde Vorbereitung erscheint der Befehl zur Einnahme der Geste (z.B. `ooooo NOW ROCK ooooo`). Der Proband nimmt daraufhin schnell und energisch die geforderte Handhaltung ein und hält diese Anspannung für die Dauer eines zweisekündigen Messfensters aufrecht. Die während dieser zwei Sekunden über die serielle Schnittstelle eintreffenden **EMG**-Messwerte der acht Kanäle werden im Skript automatisch mit dem entsprechenden Klassen-Label versehen und als `.csv`-Datei gespeichert.

Durch diese Taktung ergibt sich für ein einzelnes Gesten-Tripel ein Zeitaufwand von 9 Sekunden (3 Gesten · [2 s Aufnahme + 1 s Entspannung]). Für eine initiale Versuchsreihe wurde ein Datensatz bestehend aus 150 Gesten (50 Gesten-Tripel) aufgezeichnet, was einer Aufnahmezeit von 7,5 Minuten entspricht. In späteren Aufnahmen mit wechselnden Armband- und damit Elektrodenpositionen wurde dieser Datenbestand auf insgesamt 300 Gesten verdoppelt, um die Generalisierungsfähigkeit des Systems weiter zu erhöhen. Da jede Geste gleich oft vertreten ist, liegt hier ein perfekt balancierter Datensatz vor, was das Training und die spätere Evaluierung des Klassifikationsmodells erheblich vereinfacht.

5.3 Feature-Selektion, Modellwahl und Training am PC

Nach der erfolgreichen Erstellung eines eigenen Datensatzes erfolgte die Verarbeitung und das Modelltraining innerhalb eines Jupyter-Notebooks mithilfe der Bibliothek *scikit-learn*. Bevor das Klassifikationsmodell trainiert werden konnte, musste eine fundierte Auswahl der mathematischen Merkmale (Features) getroffen werden, da das rohe **EMG**-Signal für eine direkte Mustereerkennung ungeeignet ist.

In einem ersten Ansatz wurde testweise lediglich der Effektivwert, Root Mean Square (**RMS**), für jeden der acht Kanäle berechnet. Die mathematische Definition des **RMS** beschreibt die Signalleistung und ist wie folgt definiert, wobei N die Anzahl der Signal-

werte x_i im betrachteten Zeitraum ist:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

Obwohl dieser minimalistische Ansatz mit nur acht Merkmalen bereits eine überraschend solide Klassifikationsgenauigkeit erzielte, wurde der Feature-Vektor zur Steigerung der Robustheit um ein zweites zeiteffizientes Merkmal erweitert: den mittleren Absolutwert, Mean Absolute Value (**MAV**). Der **MAV** dient als Maß für die mittlere Intensität der Muskelaktivierung im betrachteten Zeitraum und berechnet sich nach der Formel

$$\text{MAV} = \frac{1}{N} \sum_{i=1}^N |x_i|$$

Durch die Kombination beider Merkmale über ein Analysefenster von zwei Sekunden entstand pro Datenpunkt ein eindimensionaler Feature-Vektor mit 16 Werten (8 Kanäle mit je 2 Features). Ein entscheidender Vorteil dieser beiden Merkmale liegt in ihrer inhärenten Positionsunabhängigkeit — und damit Zeitinvarianz — innerhalb des Analysefensters. Da sowohl bei der Berechnung des **RMS** als auch des **MAV** über die gesamte Fensterlänge gemittelt wird, hat die exakte zeitliche Platzierung einer Muskelkontraktion keinen Einfluss auf das Ergebnis. Dies ist für die kontinuierliche Echtzeit-Klassifikation von zentraler Bedeutung: Da das System fortlaufen in festen Intervallen klassifiziert, bleibt die Erkennung einer Geste robust. Dies ist unabhängig davon, ob die Aktivierung zu Beginn, in der Mitte oder am Ende des aktuellen Zeitfensters auftritt.

Bezüglich der Signalparameter wurde eine bewusste Entscheidung getroffen: Da der Fokus nun vollständig auf den eigenen Daten lag wurde die künstliche Normierung auf den Wertebereich von $[-128, 127]$ verworfen, welche in **Abschnitt 5.1** für die Komartibilität mit dem *Myo-Dataset* eingeführt worden war. Diese Einschränkung hätte die Datenbasis für den Klassifikator aufgrund eines kleineren Wertebereichs unnötig vergrößert. Das Herunterskalieren (Downsampling) der Abtastrate von 1000 Hz auf 200 Hz wurde hingegen beibehalten. Bei einer Fenstergröße von zwei Sekunden reduziert sich die Anzahl der mathematisch zu verarbeitenden Werte pro Kanal dadurch von 2000 auf 400 Samples, ohne dass ein signifikanter Verlust an klassifikationsrelevanten Signalanteilen entsteht.

Für die eigentliche Modellwahl wurde ein klassischer, dateneffizienter Machine Learning (**ML**) Ansatz einem neuronalen Netz vorgezogen. Versuche mit einer künstlichen neuronalen Netzwerkstruktur in *TensorFlow/Keras* zeigten keinen Genauigkeitsvorteil gegenüber anderer Lernverfahren, erforderten jedoch eine höhere Rechenleistung. Die finale Wahl fiel somit auf einen *Random Forest Classifier*. Bei diesem Klassifikationsmodell handelt es sich um einen Ensemble Learning-Ansatz, der auf einer Vielzahl unabhängiger Entscheidungsbäume basiert. Während des Trainingsprozesses wird jeder Baum auf einer zufälligen Teilmenge der Trainingsdaten sowie mit einer zufälligen Auswahl an Merkmalen aufgebaut. Für die eigentliche Klassifikation einer unbekannten Geste durchläuft der extrahierte Feature-Vektor parallel alle Bäume des *Waldes*, wobei jeder Baum eine

eigenständige Klassenvorhersage trifft. Das finale Ergebnis des Gesamtsystems wird anschließend per Mehrheitsentscheid ermittelt, indem die Geste mit den meisten Stimmen ausgewählt wird. Diese Struktur macht einen Random Forest robust gegen Overfitting und Ausreißern in den **EMG**-Signalen, während er gleichzeitig eine extrem geringe Rechenkomplexität in der Inferenz aufweist.

Das Training wurde auf Basis des im vorherigen **Abschnitt 5.2** erstellten Datensatzes mit einem klassischen Train-Test-Split von 80 zu 20 durchgeführt. Aufgrund der perfekt balancierten Klassenverteilung innerhalb des Datensatzes wurde die *Accuracy* (Genauigkeit) als primäre und hinreichende Evaluationsmetrik gewählt, welche auf Validierungsdaten eine hervorragende Genauigkeit von ca. 90 % erreichte.

5.4 Firmware-Implementierung des Klassifikationsmodells

Nachdem das Random Forest-Modell in Python erfolgreich trainiert wurde, musste die gesamte Signalverarbeitungs- und Entscheidungskette für den Echtzeitbetrieb auf den **Teensy 4.1** übertragen werden. Hierzu wurde das trainierte Modell mithilfe der Python library **m2cgen** (Model 2 Code Generator) in nativen C-Code übersetzt. Die resultierende über 4000 Zeilen lange Funktion **score()** benötigt keinerlei externe Bibliotheken oder Laufzeitumgebungen. Sie bildet die erlernten Entscheidungsstrukturen des Random Forests als eine Kaskade tief verschachtelter **if-else**-Bedingungen direkt im Code ab. Diese Struktur ermöglicht es dem Mikrocontroller, die Klassifikation mit sehr geringer Latenz und minimalem Speicherbedarf auszuführen, da im Gegensatz zu neuronalen Netzen keine komplexen Fließkomma-Operationen auf Vektoren (bzw. Arrays) berechnet werden müssen.

Die größte Herausforderung auf dem eingebetteten System liegt in der Koordination der unterschiedlichen Zeitdomänen und Frequenzen, in denen die Software operiert. Die gesamte Steuerung ist als deterministische Zustandsmaschine innerhalb der firmwareseitigen Hauptschleife realisiert. Den Takt des Gesamtsystems gibt die hardwarenahe Datenakquisition des **ADS1298** mit einer Abtastrate von 1000 Hz vor. Sobald ein neuer Datenpunkt über den **SPI**-Bus eingelesen wurde, dekrementiert die Firmware parallel zwei Software-Zähler, um die nachgelagerten Prozesse präzise anzusteuern.

Der erste Prozess betrifft das Downsampling für die Datenpufferung. Wichtig hierbei ist, dass die komplette mathematische Signalbereinigung durch die Biquad-Filter aus **Abschnitt 4.2** bei der vollen nativen Abtastrate von 1000 Hz durchgeführt wird. Dies garantiert die bestmögliche Filterleistung und verhindert, dass hochfrequente Störanteile durch Aliasing-Effekte das Nutzsignal verschlechtern. Erst nach dieser Filterung greift das Downsampling auf 200 Hz: Bei jedem fünften Durchlauf der 1000 Hz-Schleife wird der aktuell bereinigte Wert in den Ringpuffer geschrieben. Dieser Puffer hält exakt die für das Analysefenster benötigten 400 Samples pro Kanal vor.

Der zweite Prozess regelt das Inferenz-Intervall. Obwohl das Analysefenster Daten aus zwei Sekunden umfasst wird nicht solange gewartet, bis eine neue Vorhersage getroffen wird. Stattdessen wird alle 100 Durchläufe der Hauptschleife (entspricht einer Fre-

quenz von 10 Hz bzw. alle 100 ms) eine neue Klassifikation angestoßen. Hierzu berechnet der Teensy 4.1 aus den aktuellen 400 Samples des Ringpuffers die 16 **RMS**- und **MAV**-Merkmale, wobei sich an dieser Stelle die im vorherigen **Abschnitt 5.3** beschriebene der Merkmale als erheblicher Vorteil erweist: Da die Kennwerte positionsunabhängig über das gesamte Fenster gemittelt werden, muss bei der Feature-Berechnung keinerlei Rücksicht auf den aktuellen Stand des rotierenden Ringpuffer-Pointers nehmen. Es wird schlicht das gesamte Array aufsummiert. Sofern alle acht Kanäle den empirisch ermittelten **RMS**-Schwellenwert von 35.0 überschreiten, wird der 16-Dimensionale Feature-Vektor an die generierte `score()`-Funktion übergeben und das Ergebnis als neue Geste gespeichert. Werden die Schwellenwerte nicht überschritten, gilt dies als Erkennung der Ruheposition. Das System liefert somit zehnmal pro Sekunde eine aktuelle und robuste Gestenvorhersage.

Bevor ein Klassifikationsergebnis jedoch in einen physischen Steuerbefehl für die Servomotoren der Handprothese übersetzt wird, durchläuft es eine softwareseitige Schutzlogik. Würden die Motoren ungefiltert auf jede der 10 Vorhersagen pro Sekunde reagieren, würde ein kurzes Flattern des Signals oder eine einzelne Fehlklassifikation zu einer unruhigen Bewegung der Prothese führen. Zudem droht bei häufigen, schnellen Richtungswechseln eine Überlastung der Servos.

Aus diesem Grund wurde ein zeitliches Schutzfenster von einer Sekunde (1 Hz) implementiert. Sobald das Modell eine neue Geste erkennt, die von der aktuellen abweicht, werden die Motoren angesteuert und ein Zähler gestartet. Für die Dauer einer Sekunde reagiert die Motorsteuerungslogik nicht auf Änderungen der erkannten Geste. Erst nach Ablauf dieser Schutzzeit werden die Motoren wieder freigegeben um auf eine neue Geste einzunehmen.

6 Evaluierung und Ergebnisse

Nach der erfolgreichen Implementierung der Klassifikationspipeline auf dem eingebetteten System wurde dieses im praktischen Einsatz evaluiert. Die nachfolgenden Abschnitte betrachten das Systemverhalten auf Basis empirischer Daten und analysieren die Interaktionslatenz, die biomechanischen Besonderheiten der Gesten-Erkennung sowie die Robustheit des Modells gegenüber veränderten Sensorpositionen.

6.1 Systemlatenz und Echtzeit-Verhalten

Eine wichtige Metrik für die Alltagstauglichkeit einer myoelektrischen Prothese ist neben der reinen Erkennungsgenauigkeit das subjektive Empfinden der Verzögerung zwischen der mentalen Intention des Nutzers und der physischen Umsetzung durch die Aktuatoren. Liegt diese Latenz hoch, wirkt das System träge und lässt sich nur schwer präzise steuern. Bei einer optimalen Positionierung der Elektroden-Armbänder weist das Gesamtsystem eine empirisch ermittelte Verzögerung von etwa 0,5 Sekunden auf.

Diese Gesamtlatenz setzt sich aus drei Sequentiellen Phasen zusammen. Da die mathematischen Features über einen gleitenden Puffer von zwei Sekunden berechnet werden,

benötigt das System eine kurze Zeitspanne, bis sich die veränderten Signal Amplituden im Mean Absolute Value (MAV) und Root Mean Square (RMS) signifikant niederschlagen. Durch die hohe Interferenzfrequenz von 10 Hz wird die daraufhin getroffene Gestenvorhersage jedoch nahezu verzögerungsfrei in den internen Systemzustand übersetzt. Den letzten Anteil der Verzögerung bildet schließlich die mechanische Trägheit der Servomotoren, welche eine kurze physische Anlaufzeit benötigen, um die Finger der Handprothese in die Zielposition zu bringen. Mit einer Gesamtlatenz von 500 ms liegt das System in einem Bereich der als angemessen flüssig wahrgenommen wird, ohne dass das Motorschutzfenster von einer Sekunde den Bewegungsfluss blockiert.

6.2 Biomechanische Signalcharakteristika und Besonderheiten

Bei genauerer Betrachtung der Klassifikationsergebnisse zeigt sich, dass die *Schere*, *Stein*, *Papier* Gesten aus Biomechanischer Sicht ideal gewählt sind. Die drei ausgewählten Gesten nutzen anatomisch komplementäre Muskelgruppen des Unterarms, was sich in markanten, leicht trennbaren Mustern über die acht Kanäle des ADS1298 widerspiegelt. Die Geste *Stein* erfordert eine volle Flexion aller Finger. Dies führt zu einer massiven, homogenen Aktivierung primär auf jenen Elektroden, die an der Innenseite des Unterarms anliegen. Aufgrund dieser eindeutigen anatomischen Charakteristik wird die Faust vom Random Forest nahezu fehlerfrei und mit maximaler Verlässlichkeit erkannt.

Im Gegensatz dazu aktiviert die Geste *Papier* primär Extensoren an der Außenseite des Unterarms, um die Finger vollständig zu strecken. Die Geste *Schere* wiederum bildet eine Mischform, bei der Zeige- und Mittelfinger gestreckt, Ring- und kleiner Finger sowie der Daumen jedoch angewinkelt werden. Hierbei werden Muskelstränge der Innen- und Außenseite simultan aktiviert. Aufgrund dieser anatomischen Überschneidungen zeigt die empirische Evaluation, dass Konfusionen des Modells am ehesten zwischen den Klassen *Papier* und *Schere* auftreten. Wenn die Streckbewegung der Schere sehr dominant ausgeführt wird, neigt der Klassifikator dazu, die Geste fälschlicherweise als offene Hand zu interpretieren. Ein weiteres Phänomen betrifft den Übergang beim Lösen der Geste *Stein* in den Ruhezustand. In dem Moment, in dem die Anspannung der inneren Beugemuskeln nachlässt, wird für einen Sekundenbruchteil oft die Geste *Schere* erkannt. Diese Fehlklassifikation resultiert vermutlich aus der ungleichmäßigen Muskelentspannung und dem damit verbundenen Einschwingverhalten des Signals im abklingenden Analysefenster. Dennoch bleibt die Gesamtperformance des Systems auf einem hohen und stabilen Niveau.

6.3 Robustheit gegenüber Sensor-Repositionierung (Spatial Shift)

Ein kritisches Phänomen bei sEMG-Systemen ist die Empfindlichkeit gegenüber dem erneuten Anlegen der Sensorik nach einer Nutzungspause. In der alltäglichen Praxis ist es für einen Nutzer kaum verlässlich möglich, die Elektroden-Armbänder nach dem Ablegen wieder exakt in der selben Position und Rotationsausrichtung auf dem Unterarm zu platzieren. Die Evaluierung dieses Szenarios ergab, dass die Genauigkeit des Modells

von den ursprünglichen 90 % auf Werte zwischen 60 % und 70 % abfällt, wenn das Armband rein nach Augenmaß und ohne technische Hilfsmittel neu positioniert wird. In diesem Fall passen die gelernten Verzweigungsschwellenwerte der Entscheidungsbäume nicht mehr perfekt zu den nun leicht verschobenen Kanälen. Gelingt eine exakte Repositionierung, bleibt die hohe Ausgangsgenauigkeit erhalten, was die Validität des Modells unterstreicht. Ein Referenzfoto hilft beim Wiederanlegen, ist jedoch nicht ausreichend um eine exakte Positionierung verlässlich zu ermöglichen.

Da eine perfekte Platzierung im Alltag unrealistisch ist, wurde untersucht wie die Genauigkeit des Systems wiederhergestellt werden kann. Es zeigte sich, dass bereits ein minimales Nachtraining (Fine-tuning) mit nur etwa 30 neuen Datenpunkten in der veränderten Position ausreicht um die Klassifikationsgenauigkeit des Modells direkt wieder auf das hohe Niveau von 80 % bis 90 % anzuheben. Dies entspricht einem kleinen Datensatz von zehn Gesten pro Klasse, welcher mit dem in [Abschnitt 5.2](#) beschriebenen Verfahren innerhalb von 90 Sekunden (1,5 Minuten) aufgenommen werden kann. Da auch das Modelltraining, der Modell-Export und die Programmierung des **Teensy 4.1** sehr zeiteffizient möglich ist, kann das ganze System von einem geübten Anwender innerhalb von etwa 3 Minuten auf die neue Armbandpositionen angepasst werden. Diese Beobachtung beweist, dass das trainierte Modell eine hohe Generalisierbarkeit gegenüber leichter Verschiebungen der Sensoren besitzt, welche durch ein effizientes Fine-tuning kompensiert werden kann. Der mobile Einsatz der Prothese ist dadurch jedoch signifikant eingeschränkt.

7 Zusammenfassung und Ausblick

Im Rahmen dieser Projektarbeit wurde die myoelektrisch gesteuerte Handprothese des Vorseminesters erfolgreich von einer Schwellenwertsteuerung zu einer intelligenten, musterorientierten Erkennung weiterentwickelt. Durch das hardwareseitige Upgrade von vier auf acht **sEMG**-Kanäle mithilfe des **ADS1298** Biosignalverstärkers konnte die räumliche Auflösung der Muskelaktivitätsmuster am Unterarm des Anwenders entscheidend erhöht werden. Die Implementierung einer digitalen Filterkaskade direkt auf dem **Teensy 4.1** ermöglichte eine hochgradig effektive Online-Bereinigung der Signale von Netzbrummen und Bewegungsartefakten bei einer nativen Abtastrate von 1000 Hz.

Auf dieser bereinigten Datenbasis wurde eine effiziente Edge AI-Pipeline realisiert, die durch ein Downsampling auf 200 Hz und ein zeitinvariantes Feature-Engineering im Zeitbereich (**RMS** und **MAV**) geprägt ist. Durch den Einsatz des Tools **m2cgen** konnte ein in Python trainierter *Random Forest Classifier* als tief verschachtelte **if-else**-Struktur nativ in die Firmware eingebunden werden. Im autarken Echtzeitbetrieb auf dem Mikrocontroller erzielte dieses System bei der Unterscheidung der Gesten *Schere*, *Stein*, *Papier* eine hervorragende Validierungsgenauigkeit von circa 90 %. Es konnte eine empirische Interaktionslatenz von etwa 5,0 Sekunden erreicht werden.

Das System demonstriert eindrucksvoll, dass mit mathematisch optimierter Signalverarbeitung eine robuste und flüssig reagierende myoelektrische Gestensteuerung auf eingebetteten Systemen realisierbar ist.

7.1 Zukünftige Optimierungs- und Erweiterungspotenziale

Trotz der stabilen Funktionalität des Prothesensystems eröffnen die gewonnenen Erkenntnisse vielversprechende Ansätze für zukünftige Forschungsarbeiten. Ein zentraler Aspekt der Systemdynamik betrifft den zeitlichen Verlauf der extrahierten **RMS** und **MAV** Merkmale. Empirisch lässt sich beobachten, dass diese Kennwerte bei einer Muskelkontraktion langsam ansteigen und nach dem Lösen der Geste ebenso verzögert absinken. In zukünftigen Untersuchungen sollte analysiert werden, ob eine gezielte Verkürzung des aktuellen Analysefensters von zwei Sekunden (400 Samples bei 200 Hz) diesen Signalübergang beschleunigen kann. Das Ziel besteht darin, die Systemlatenz von derzeit circa 500 ms weiter zu minimieren, ohne dabei die Stabilität der Gestenerkennung zu gefährden.

Ein weiterer softwareseitiger Ansatzpunkt liegt in der tieferen Analyse der Kanalrelevanz des aktuellen 8-Kanal-Setups. Hierbei geht explizit nicht um das Einsparen von Pins oder Rechenleistung, sondern um ein fundiertes anatomisches Verständnis der Signalkomponenten. Durch die Deaktivierung einzelner Kanäle soll erforscht werden, welche spezifischen Elektrodenpositionen die trennschärfsten Informationen für die Mustererkennung liefern und inwiefern sich die Signalcharakteristika einzelner Kanäle voneinander unterscheiden.

Darüber hinaus bildet die Weiterentwicklung der Klassifikationslogik Raum für Optimierungen. Im aktuellen System wird die Ruheposition über eine **RMS**-Schwellenwertlogik erkannt. Ein theoretischer Alternativansatz wäre die Integration des Ruhezustands als eigene kontinuierlich klassifizierte Modellklasse im Random Forest. Auf basis der vorliegenden Ergebnisse ist dieser Ansatz jedoch kritisch zu betrachten, da die aktuelle Schwellenwertlogik bereits robust funktioniert. Die Einführung einer permanent aktiven Ruheklasse würde zudem gravierende Probleme bei der Datensatz-Balancierung aufwerfen, da der Zustand der Muskelentspannung im Alltag zeitlich unbegrenzt auftritt und sich kaum als gleichwertiges Äquivalent zu den klar abgegrenzten Aktionsklassen in ein balanciertes Trainingsschema integrieren lässt.

7.2 Probanden-Kompatibilität und mobiles Training

Für eine echte Alltagstauglichkeit muss sich das System zudem von der Abhängigkeit externer PC-Systeme lösen. Derzeit führt der *Spatial Shift* beim Wiederanlegen der Elektroden-Armbänder zu einem spürbaren Einbruch der Klassifikationsgenauigkeit, welcher ein kurzes Nachtraining am PC erfordert. Zukünftige Arbeiten sollten daher die Möglichkeit einer Implementierung von embedded **ML**-Algorithmen direkt auf dem **Teensy 4.1** untersuchen. Ein solches On-Device-Training würde es dem Anwender erlauben, das notwendige Fine-Tuning nach dem Anlegen des Armbands komplett autark über eine integrierte Routine an der Prothese durchzuführen, wodurch die Mobilität und Alltagstauglichkeit des Gesamtsystems signifikant gesteigert wird.

Abschließend ist die Durchführung einer umfangreichen Probandenstudie unerlässlich, um die Allgemeingültigkeit des entwickelten Systems zu validieren. Hierbei steht die Fragestellung im Vordergrund, inwieweit sich die gelernten Klassifikationsstrukturen auf andere

Individuen übertragen lassen. Zu untersuchen ist, ob das Modell für jede Person grundlegend neu trainiert werden muss, oder ob ein universeller, anatomischer *Basis-Wald* generiert werden kann, der als Ausgangspunkt für ein schnelles, probandenspezifisches Transfer Learning dient.

Parallel zu den Fragen der Robustheit und Portabilität stellt die Erweiterung des Klassenspektrums ein essenzielles Ziel dar, um den klinischen und praktischen Nutzen der Prothese im Alltag zu erweitern. Während sich die *Schere, Stein, Papier*-Gesten in dieser Projektphase als hervorragendes Modell für eine wissenschaftliche Machbarkeitsstudie erwiesen hat, bieten sie für den Anwender im täglichen Leben nur einen eingeschränkten Nutzwert. Die stabil funktionierende Architektur dient daher als direktes Fundament, um zeitnah von der Erkennung einfacher Handhaltungen zu einer komplexen Einzelfingersteuerung und differenzierten Greif-Koordination überzugehen. Dafür bietet es sich an, die öffentliche sEMG-Datenbank *Ninapro* mit einzubeziehen, wobei insbesondere der Datensatz *DB5* einen vielversprechenden Ansatzpunkt darstellt [5, 7]. Auf diese Weise kann das System von einer experimentellen Steuerung zu einer alltagstauglichen, funktionellen Handprothese ausgebaut werden.

Abbildungsverzeichnis

1	Vergleich von ungefiltertem und gefiltertem EMG-Signal.	8
---	---	---

Tabellenverzeichnis

Literatur

- [1] Purushothaman Geethanjali. „Myoelectric control of prosthetic hands: state-of-the-art review“. In: *Medical Devices: Evidence and Research* (2016), S. 247–255.
- [2] WOI Kellner. *Verschiedene Arten der Handprothese*. <https://www.woi-kellner.de/verschiedene-arten-der-handprothese/>. Abgerufen am 22.06.2026.
- [3] Roberto Merletti und Dario Farina. *Surface electromyography: physiology, engineering, and applications*. John Wiley & Sons, 2016.
- [4] Michidk. *Myo Dataset GitHub Repository*. <https://github.com/michidk/myo-dataset>. Abgerufen am 22.06.2026. 2026.
- [5] Ninapro Team. *Ninapro Datasets*. <https://ninapro.hevs.ch/>. Abgerufen am 06.07.2026. 2026.
- [6] Ottobock SE & Co. KGaA. *Myo Plus TR*. <https://www.ottobock.com/de-de/product/13E520-61077>. Aufgerufen am 04.07.2026. 2026.
- [7] Stefano Pizzolato u. a. „Comparison of six electromyography acquisition setups on hand movement classification tasks“. In: *PLOS ONE* 12.10 (2017), e0186132. DOI: [10.1371/journal.pone.0186132](https://doi.org/10.1371/journal.pone.0186132).
- [8] Isabell Zeitler und Anton Laubhan. *Entwicklung einer myoelektrisch gesteuerten Handprothese*. Projektarbeit eingebettete Systeme 2 (Unveröffentlicht). Projektbericht des Vorsemesters. Hochschule für angewandte Wissenschaften Landshut, Fakultät Elektrotechnik und Wirtschaftsingenieurwesen, Jan. 2026.

Übersicht verwendeter Hilfsmittel

[Übersicht und Dokumentation der genutzten KI-Werkzeuge und Prompts gemäß den Richtlinien]